Avni Sanghvi

DSC 450 | Databases for Analytics |Spring 2026 | Assignment 1

Part A – Conceptual Database Foundations

Q1) Explain (any) two different responsibilities of database systems that go beyond basic functionality of storing data. For each, give a concrete example (e.g., based on a university, healthcare database, or e-commerce setting).

→ Let us consider the scenario of a university database. Two responsibilities of database, beyond the functionality of storing data are:

i. **Constraints Enforcement:** The implementation of a database includes setting up rules for what kind of data can be entered, setting up access controls which define the power/access given to different users.
For example, a sequence of strings can only be entered under 'name' column, or the 'grades' column will accept only a sequence of up to 2 characters (for instance, A/A+). Also, no grades can be updated for a student who is not enrolled in that course.
These constraints ensure that database is not simply storing data, but it is keeping the data **valid** and **meaningful**.

ii. **Concurrency of actions:** The university database holds data of multiple students. There are situations where multiple students are trying to work on the same kind of action. For example, while enrolling in a course, multiple students are trying to enroll in subject A at the same time. At some point, the system might read 'seat left: 1' for all students left to enroll. The database will process the request of first student in queue and then it will put the rest into waiting state.
The database keeps a check that the simultaneous actions are not interfering with each other.

Q2) Consider the following schema describing a restaurant platform:
Customer(customer_id, name, email)
Restaurant(restaurant_id, name, is_open)

Order(order_id, customer_id, restaurant_id, total_price, status)

OrderItem(order_id, item_name, quantity, price)

Identify and explain

1) primary keys (column or a set of columns that identify each record)

→ **Customer**: customer_id
This column uniquely identifies the data of one customer. Hence, it is the primary key of the given schema.
Restaurant: restaurant_id

The column restaurant_id column uniquely identifies the data of a restaurant. Hence, it is the primary key of the given schema.

Order: order_id

The column order_id column uniquely identifies the data of one particular order placed. Hence, it is the primary key of the given schema.

OrderItem: order_id and item_name

The two columns, order_id and item_name , in composition, identify the unique record in the schema. Those two columns individually won't work in identifying the row uniquely.

2) foreign key relationships (foreign key column and what it references)

→ There is **no foreign key** column in the **Customer** table and **Restaurant** table.

→ In the schema **Order**, the columns **customer_id** and **restaurant_id** are foreign key columns. These are the primary key columns in tables, Customer and Restaurant, respectively, and so, are foreign key columns to this schema. These keys will help in correctly linking the order with the correct customer and the correct restaurant details with the order details.

→ In the schema **OrderItem**, the column **order_id** is the **foreign key** column because it is the primary key column in the **Order** table. This key will link the details of the order item with the data of the order placed for that item.

3) domain constraints (you can make any reasonable assumptions)

→ **Customer (customer_id, name, email):**

- 'customer_id' will hold unique values. It will be of integer type and cannot be NULL.
- 'name' will hold character values only, up to 50 characters max.
- 'email' will be of string data type, with a combination of letters and numbers. It must contain one '@' and one '.' sign in it. It should also follow a pattern of _@_._

→ **Restaurant (restaurant_id, name, is_open):**

- 'restaurant_id' will hold unique values. It will be of integer type and cannot be NULL.
- 'name' will hold character values only, up to 50 characters max.
- 'is_open' will accept only two values – y/n, signifying yes/no.

→ **Order (order_id, customer_id, restaurant_id, total_price, status)**

- 'order_id' will hold unique values. It will be of integer type and cannot be NULL.
- 'customer_id' and 'restaurant_id' values will match the values in the 'Customer' and 'Restaurant' table respectively.
- 'total_price' will hold decimal values with two decimal places and it must be >=0.
- 'status' will hold string values. It can hold values like 'Order Placed', 'Preparing to ship', 'Ready to ship', 'Shipped', 'Out for delivery' and 'Delivered'.

→ **OrderItem(order_id, item_name, quantity, price)**

- ‘order_id’ values will match the values in ‘order_id’ column of the ‘Order’ table.
- ‘item_name’ will hold string values.
- ‘quantity’ value is of integer type. It cannot be zero or negative and neither in decimals.
- ‘price’ will be of float type. It will never be negative, and it can come down to zero.

4) semantic constraints (business rules – e.g., quantity must be >= 1).

→ An Order cannot be created if the Restaurant.is_open status for that restaurant_id is currently 'n’.

→ The Order.total_price must equal the mathematical sum of (quantity * price) for all corresponding rows in the OrderItem table.

→ A specific item_name should only appear once per order_id. If the customer wants more, the quantity attribute must be updated rather than creating a duplicate row.

→ An Order.status cannot jump from ‘Order Placed’ straight to 'Delivered'; it must logically pass through 'Preparing to ship' or 'Out for Delivery'.

Q3) You are designing a database for an online marketplace where sellers list items for sale. Consider a single relation:

Listing (listing_id, item_name, seller_id, seller_name, seller_email, price)

Assumptions:

- Each listing has exactly one seller

- Each seller has exactly one name and one email

- A seller may have many listings

Answer the following:

1) List all functional dependencies that logically follow from the stated assumptions

→ A functional dependency, X -> Y exists if, for every instance of X, the value of Y can be determined.

→ The functional dependencies that are observed here are:

- Since listing_id is unique, it will help to determine every other column value in the table.
  listing_id -> item_name, seller_id, seller_name, seller_email, price
- Each seller has exactly one name and one email. So,
  seller_id -> seller_name
  Seller_id -> seller_email
- listing_id -> seller_id -> seller_name

2) Explain one type of redundancy that may occur in this design.

→ The seller details of name and email are linked to the seller_id and all the details are stored in the same table. This might create Data Redundancy in the table.

→ Since a seller can have multiple listings, their name and email will be repeated for every item they post.

3) Describe one problem that could arise in practice if a seller changes their email address (hint: consider the situation in which the seller has already made multiple listings before changing their address).

→ There is only one table and there is **no separate table** to store the seller's details.

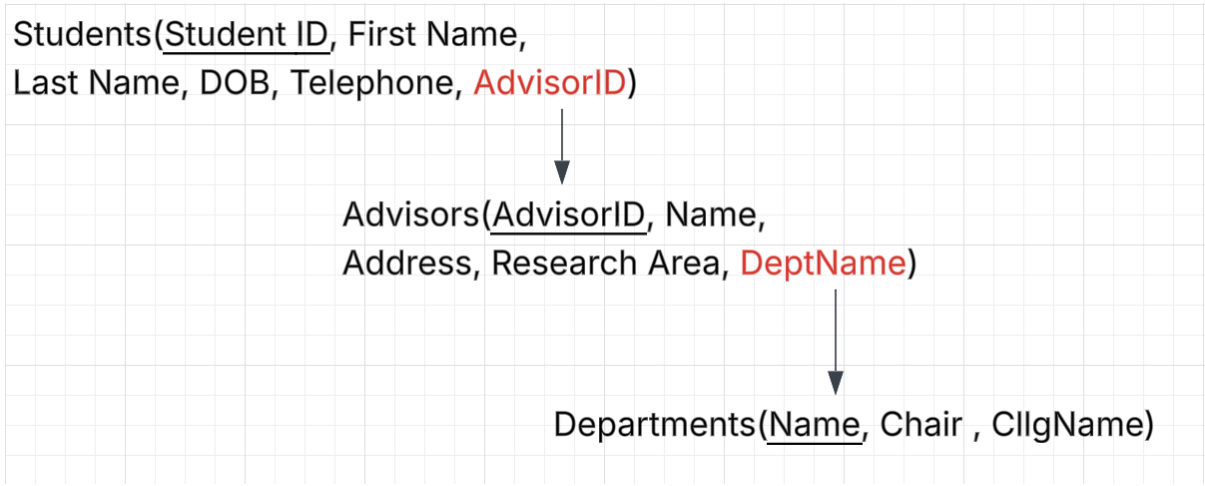
→ In this situation, if multiple listings are already made and then the email is changed, the database **must update all the rows affecting it simultaneously.** However, changing multiple rows all together might not end up successful in following scenarios:

- If the database or the application gets **crashed** in middle of the updating process, there would be some rows left unchanged.
- If any user is searching for that buyer's contact information at the time of updating, it is possible it gets the old information of the buyer.
- If a customer service agent searches for the seller using the 'new' information, they might fail to find the listings that haven't been updated yet, leading to the false conclusion that those items don't exist or belong to someone else.

In all possible scenarios, there is a risk that data might become **inconsistent**, making it **corrupt** or **out of sync**.

Q4) Define and show a relational schema for students, student advisors, and advisor departments

- **Students have StudentID, First Name, Last Name, DOB, Telephone and a reference to their advisor**
- **Advisors have ID, Name, Address, Research Area, and a reference link to their Department**
- **Departments have Name, Chair, College (identified by Name)**



→ The foreign key columns of each table are written in 'red' color.

Part B – Python and Data

Q5) You are given student data as a list of dictionaries:

Students = [{"student_id": 1, "name": "Bob", "email": "bob213@myu.edu", "year": 2},
{"student_id": 2, "name": "Alice", "email": "alice223129@myu.edu", "year": 5},
{"student_id": 2, "name": 17, "email": "charlie2@myu.edu", "year": 3}]

Answer the following:

1) Write a python function that checks the primary key constraint on student_id.

2) Write a function that checks the following domain constraints: year must be in the domain (1, 2, 3, 4). Name must be a string of 20 characters or shorter. Email username must be a maximum of 8 characters. Your function should report all violations, not just the first one it detects.

Q6) Given that a student must enroll in at least 1 course and may enroll in at most 4 courses, consider the following data:

Enrollments = [(1, "DSC 450"), (1, "CSC 402"), (1, "IS 451"), (1, "DSC 578"),
(1, "CSC 444"), (1, "DSC 501"), (3, "CSC 555")]

Answer the following:

1) Write a function that detects violations of these two constraints

2) Make sure your function returns the list of (student_id, number_of_courses) for violating students

Solution for Q5 and Q6:

week-1/hw-1/assignment-1.py

```python
# Problem 5

Students = [ {"student_id": 1, "name": "Bob", "email": "bob213@myu.edu", "year": 2},
{"student_id": 2, "name": "Alice", "email": "alice223129@myu.edu", "year": 5},
{"student_id": 2, "name": 17, "email": "charlie2@myu.edu", "year": 3} ]

# Problem 5.1
# Function Definition for primary key constraints
def check_primary_key_constraints(students):
    seen_ids = set()

    for student in students:
        student_id = student.get("student_id")

        if student_id in seen_ids:
            return "The data do not follows the primary key constraints"

        seen_ids.add(student_id)

    return "The data follows the primary key constraints"

#Problem 5.2
# Function Definition for domain constraints
def check_domain_constraints(students):
    for student in students:
        sid = student.get("student_id")
        violations = []

        # year check
        year = student.get("year")
        if year not in (1, 2, 3, 4):
            violations.append("incorrect year")

        # name check (type + length)
        name = student.get("name")
        if not isinstance(name, str):
            violations.append("name is not a string")
        else:
            if len(name) > 20:
                violations.append(f"name length is {len(name)}")

        # email check (username length only)
        email = student.get("email")
        if isinstance(email, str) and "@" in email:
            username = email.split("@")[0]
            if len(username) > 8:
                violations.append(f"email username length is {len(username)}")
        else:
            violations.append("invalid email format")

        # print results
        if violations:
```

```python
            print(f"Student with ID {sid} violations: {', '.join(violations)}")
        else:
            print(f"Student with ID {sid} follows all domain constraints")

# Function Call for primary key constraints
result = check_primary_key_constraints(Students)
print(result)

# Function Call for domain constraints
check_domain_constraints(Students)

# Problem 6

Enrollments = [ (1, "DSC 450"), (1, "CSC 402"), (1, "IS 451"), (1, "DSC 578") , (1, "CSC 444"),
                (1, "DSC 501"), (3, "CSC 555")]

# Function Definition to check Enrollment Constraints
def check_enrollment_constraints(enrollments):
    course_count = {}

    for student_id, _ in enrollments:
        if student_id in course_count:
            course_count[student_id] += 1
        else:
            course_count[student_id] = 1

    violations = []
    for student_id, count in course_count.items():
        if count < 1 or count > 4:
            violations.append((student_id, count))

    return violations

# Function call for enrollment constraints check
enroll_violations = check_enrollment_constraints(Enrollments)
print(f"Student with ID: {enroll_violations[0][0]} has enrolled in: {enroll_violations[0][1]} courses")
```